

Deep ML from Scratch: World Models Track

A 24-project curriculum from linear regression to DreamerV3 and DIAMOND. Every project is a component that appears inside a real world-model paper.

How to use this list

Work through the projects in order. Phases 1-3 are NumPy-from-scratch where you write backprop and BPTT by hand. Phases 4-6 use PyTorch but you still implement the model architectures yourself (no nn.LSTM, no nn.Transformer). Pacing target: 2-5 days per project in Phases 1-5; 2-6 weeks per project in Phase 6 where training time dominates.

Phase 1 - Foundations (NumPy)

1. Linear + logistic regression

Gradient descent on a real dataset. Both regression and binary classification. The universal pattern: forward, loss, backward, update.

2. MLP + backprop on MNIST

Two hidden layers, write backprop by hand, no autograd. Target >97% test accuracy. This single project teaches more than the next five combined.

3. Tiny autograd engine

Karpathy micrograd style. A Value class with .backward(). Then re-do MLP using it. Now you truly understand what PyTorch is doing under the hood.

Phase 2 - Vision (PyTorch)

4. CNN on CIFAR-10

Conv2d, pooling, fully connected head. Your first visual encoder. Target ~80% accuracy.

5. ResNet

Implement the residual block yourself. Train ResNet-18 on CIFAR-10. Skip connections are the most important architectural idea in deep learning.

Phase 3 - Sequences

6. RNN from scratch in NumPy with BPTT

Character-level language model on tiny Shakespeare. Manual backpropagation through time. Karpathy's min-char-rnn is the reference.

7. LSTM cell from scratch

PyTorch tensors but no nn.LSTM. All four gates, by hand. Train on the same char-LM task and compare to the vanilla RNN.

8. Mixture Density Network

Predict a Gaussian mixture instead of a point estimate. Required for the MDN-RNN dynamics model in World Models 2018.

9. Self-attention + tiny GPT

nanoGPT style. Build self-attention, multi-head, positional encoding, the transformer block, and a tiny

World Models Track

GPT on tiny Shakespeare.

10. Vision Transformer (ViT)

Patches + transformer on CIFAR-10. Used in newer transformer-based world models.

Phase 4 - Representation & Generation

11. Autoencoder

MNIST reconstruction. Encoder + decoder + bottleneck. Visualise the latent space.

12. VAE

Reparametrisation trick, ELBO loss, latent space visualisation. This is the encoder used in Ha & Schmidhuber's World Models.

13. VQ-VAE

Discrete latent codes via vector quantisation. Used in IRIS, Genie, and most modern transformer world models.

14. DDPM diffusion

Small U-Net denoiser. Forward noising, reverse denoising. Train on MNIST.

15. Conditional / action-conditioned diffusion

Extend DDPM with class conditioning and classifier-free guidance. Action-conditioning is what DIAMOND uses for world model dynamics.

Phase 5 - Reinforcement Learning

16. Gym envs + replay buffer toolkit

Reusable wrappers: frame stacking, action repeat, reward clipping, a circular replay buffer. You will reuse this in every later project.

17. Q-learning on gridworld

Tabular. Bellman equation, TD update, epsilon-greedy exploration.

18. DQN on CartPole

Deep Q-network with target network and replay buffer. The bridge from tabular RL to deep RL.

19. REINFORCE + Actor-Critic

Policy gradient methods. CartPole. Compare variance.

20. PPO on LunarLander

Proximal Policy Optimisation. The standard policy used inside every modern world model.

Phase 6 - World Models (the goal)

21. Ha & Schmidhuber World Models (2018) on CarRacing

VAE + MDN-RNN + CMA-ES controller. The canonical from-scratch world model. Three separate training stages. Plan ~2 weeks.

22. PlaNet / Dreamer-v1

Recurrent State-Space Model (RSSM). Latent-space dynamics and planning by imagining rollouts. Plan ~3 weeks.

23. IRIS (transformer + VQ-VAE world model)

World Models Track

Transformer dynamics over discrete VQ-VAE tokens. Trained on Atari with sample efficiency. Smaller jump from your earlier work than Dreamer.

24. **DreamerV3 (DIAMOND as stretch)**

Actor-critic learned entirely inside imagined latent rollouts. Solves Atari, DMC, Minecraft with the same hyperparameters. DIAMOND replaces the dynamics model with diffusion - read it if not reproducing. Plan ~4-6 weeks.

World Models Track

Why this list works for world models

A world model has three parts: a representation/encoder, a dynamics model, and a policy. This curriculum builds each piece in isolation before assembling them.

- Encoder: AE (#11) -> VAE (#12) -> VQ-VAE (#13)
- Dynamics: LSTM (#7) -> MDN (#8) -> Transformer (#9) -> Diffusion (#14, #15)
- Policy: DQN (#18) -> Actor-Critic (#19) -> PPO (#20)
- Integration: World Models 2018 (#21) -> Dreamer (#22) -> IRIS (#23) -> DreamerV3/DIAMOND (#24)

Realistic timeline

Phases 1-5 (projects 1-20): ~3-4 months at 2-5 days each.

Phase 6 (projects 21-24): ~3-4 months. Training time and debugging dominate, not coding.

Total: 6-9 months end-to-end working hard and fast.

What was deliberately cut

SVMs, decision trees, naive Bayes, k-means, HMMs, GMMs, GANs, normalising flows, MLOps. Important elsewhere but not on the critical path to world models. Add later if needed.